

Process Monitor



L. KARTHIKEYAN

This utility program is used for monitoring the working of a real-time file system, registry and process or thread activity in a PC. It helps early fault-detection and immediate identification of out-of-limit variables for real-time process control, giving users the possibility to remedy potential failures before these occur. It can be used as a tool for system troubleshooting and malware hunting.

Operating system processes

When a process needs to store temporary data, it can request a memory block from the central memory pool by way of dynamic memory-allocation. However, the total available memory is limited at any point of time. If one process eats up all the free memory, then other processes will not be able to get their required memory. Implications of a memory-starved process can lead to shutdown or unexpected crash. Clearly, none of these results are desirable to a programmer, so the processes should never reach such a state or the system itself should not starve for memory-allocation.

It is the responsibility of each process to free dynamically-allocated memory when the process is completed. The memory, when freed, goes again to the central pool, where it can be reallocated to another process on demand. When a process dynamically allocates memory and does not free that memory after use, that process heads to memory leak.

Memory leaks add up over time and, if these are not cleaned, the system eventually runs out of memory. For example, we come across an out-of-memory situation on Linux

machines when memory requests gets higher than the available free pool. This is typically because of the fact that the unused memory was not released over time or processes recursively request for more memory blocks.

Here comes the need to monitor dynamic memory requests of processes. Especially, on a development system, memory requests monitoring, as well as memory-usage pattern at different time quantum, becomes vital to capture leaks.

How a process monitor works

Processes use malloc, realloc and free function calls for dynamic memory requests. These functions are actually like a wrapper to real `__libc__` memory function calls, which does the real allocation.

Since we want to monitor and log details of these requests like requested block size in bytes, allocated region (i.e., pointer address), etc, we have to hook the control flow in between these wrapper functions and the real `__libc__` procedure call. To demonstrate this, we have a daemon (or Linux process), say, `pmd`, where we use malloc and realloc function calls for requesting memory blocks (and releasing these) periodically.

We have written a set of intermediate functions to hook the flow. From inside the hook function, we log details of the request. For logging purpose, we use a kernel module, `pmm` and a character device for communication. The kernel module becomes necessary when we want to understand and process the memory-usage footprint of a process at any point of time. Based on the memory footprint against time, the user can decide whether the process is a leaking memory or not. I leave this

part of kernel programming open for readers to develop according to their requirements.

For implementation, the kernel module simply does the following:

1. Register a character device for the user space process to communicate
2. Use the device for logging memory-allocation requests from the user space daemon
3. Use `/proc` file system to display logs

Software program

The program includes following packages: `pmd.c` (Linux user space daemon), `pmm.c` (Linux kernel module), `pmm.h` (common header file), and `makefile` (Linux kernel makefile).

User space process. The `pmd` is a user space process (or daemon) that runs an endless loop. The malloc and realloc are invoked for dynamic memory-allocation requests. We have added some functions for the purpose of hooking in-between. The hook functions, which we introduced, log memory request details by way of character device ioctl (input/output control), registered by our kernel module.

```
/* This is our daemon's main function */
int main( int argc, char *argv[]) {
    char *chr;
    daemonize(); /* Makes this Linux process
as daemon */

    openlogs(); /* This enables the hooking
of local memory request functions for
logging */
    while (1) { /* Endless loop; daemon's
main execution loop */
        sleep(10);
        chr = (char *)malloc(sizeof(char)*100);
        /* malloc call */
        sleep(10);
        chr = (char *)realloc(chr,
sizeof(char)*200); /* realloc call */
        sleep(10);
        free(chr); /* free call */
    }
    closelogs(); /* This disables the
```